Exploring and Enhancing Placement of Intrusion Detection Systems in RPL: A Federated Learning-Based Approach

Mohammadilyas Patel (CS23B1078), **Shaik Sifat Ahmed** (CS23B1035), **Abhinay** (CS23I1009)

Team 19 • Department of Computer Science and Engineering

Indian Institute of Information Technology, Design and Manufacturing, Kancheepuram, Chennai 600127, India

Project Repository: https://github.com/mdilyas-27/cns_project_team_19

Abstract*—Internet of Things (IoT) devices often run on small batteries and slow wireless links. In these low-power and lossy networks (LLNs), the Routing Protocol for Low-Power and Lossy Networks (RPL) builds a tree called a DODAG. However, legitimate sensor nodes can get captured by internal adversaries. Most intrusion detection systems (IDS) assume a central monitor at the root can inspect all network traffic. In practice, this assumption breaks down. In this course project, we study where an IDS should be placed in an RPL network. We evaluate three setups across 31,626 Cooja simulation runs in a 10-level grid topology: root-only monitoring (CIDwL), global streaming from multiple nodes (CIDwG), and distributed voting (DCID). Our findings show that root-only monitoring fails when attacks occur deep in the network. For a Blackhole attack, detection accuracy falls from 98.8% at level 1 down to 51.5% at level 8. On the other hand, streaming raw data from 9 monitors floods the radio channel with 378 kB of traffic per round. To solve this dilemma, we evaluate FedID, which uses Federated Learning with XGBoost. FedID trains local trees and averages them at the root. It needs zero bytes of communication during testing. By sharing a small 10% slice of local data across nodes, FedID overcomes non-IID traffic shifts and reaches 95.7% accuracy on Blackhole attacks alongside 99.60% on the benchmark CICIoT2023 dataset. Finally, we design DR-FedID, an extension that adds dynamic monitor rotation, Multi-Krum defense against poisoned model updates, and local differential privacy.

***Index Terms*—RPL, Low-Power and Lossy Networks (LLNs), Intrusion Detection System (IDS), Placement Strategy, Federated Learning, XGBoost, Non-IID Traffic, Byzantine Resilience, IoT Security.**

I. INTRODUCTION

The Internet of Things (IoT) connects billions of smart devices worldwide. Many of these devices operate under strict hardware limits. They use 16-bit microcontrollers, have less than 10 KB of RAM, and run on small batteries. To connect these constrained nodes, the IETF standardized RPL (RFC 6550) [1]. RPL sets up a tree-like Destination-Oriented Directed Acyclic Graph (DODAG) rooted at a border router. It uses objective functions like MRHOF to pick parents based on link quality.

While RPL works well for routing, it was not built with strong security in mind [2]. If an adversary captures a legitimate sensor node, it gains the node's cryptographic keys. The attacker can then inject false routing packets or silently drop data packets. Cryptographic checks at the link layer cannot catch these insider threats.

Intrusion Detection Systems (IDS) serve as the next line of defense. However, most prior research has focused almost entirely on testing different machine learning models, like SVMs, Random Forests, or deep neural networks [4]-[6]. These works usually assume that a central monitor at the root node can inspect all packets across the entire network. In a real multi-hop sensor network, that assumption is unrealistic. Nodes far away from the root communicate over lossy wireless channels. Packets dropped deep in the network never reach the root sink at all.

This brings up a practical question: where should we actually place the IDS engines in an RPL network? In this course project for Computer Networks and Security (CNS), we evaluate the placement trade-offs analyzed by Yilmaz et al. [3]. We focus on three core questions:

1) How much does detection drop as the attacker moves further from the root?
2) How many monitoring nodes are needed if we stream data globally, and what is the cost?
3) Can distributed voting reduce message overhead without causing too many false alarms?

We also evaluate FedID, a federated learning framework using XGBoost that eliminates testing overhead. Finally, we formulate a novel extension called Dynamic and Resilient FedID (DR-FedID) for future iterations.

II. RELATED WORK

Many researchers have studied machine learning for IoT intrusion detection. Verma and Ranga [4] built ensemble classifiers for RPL attacks. Napiah et al. [5] proposed CHA-IDS to inspect compressed 6LoWPAN headers. Shukla [6] used K-means and decision trees to detect wormhole attacks. While these systems achieved high reported scores, they all relied on centralized packet collection.

Sending all raw network packets to a central root node causes major issues in lossy networks. Raza et al. [7] noted that transmitting continuous feature vectors quickly drains node batteries and congests 802.15.4 wireless channels. Distributed trust systems offer one alternative [8], [9]. In these systems, neighbor nodes monitor each other. But computing trust tables on small microcontrollers takes up valuable memory and compute cycles.

Federated Learning (FL) offers a promising middle ground [10]. Instead of streaming raw telemetry, nodes train local models on their own data and only share model parameters with the root. While FL has been applied to general IoT tasks [11], its behavior in multi-hop RPL topologies was not clearly understood until the recent study by Yilmaz et al. [3]. Our project builds directly on their empirical findings.

III. PROBLEM FORMULATION AND METHODOLOGY

A. Routing Protocol and Threat Formulation

RPL networks use four ICMPv6 control messages to maintain topology: DODAG Information Object (DIO) for downward route advertisement, DODAG Information Solicitation (DIS) to request DIOs, Destination Advertisement Object (DAO) for upward routes, and DAO-ACK. Each node calculates a scalar Rank value. Rank increases downward as nodes get further from the root.

We examine seven insider attacks commonly found in RPL networks:

1) *Decreased Rank (DR):* An attacker advertises a fake lower Rank. Downstream nodes switch to the attacker as their preferred parent, allowing it to intercept traffic.
2) *Increased Version (IV):* An attacker sends out a higher DODAG version number. This forces the entire network to rebuild its routing tree, wasting energy.
3) *Blackhole (BH):* The compromised node simply drops every data packet it receives.
4) *Selective Forwarding (SF):* The attacker drops some packets while forwarding others, making detection harder.

5) *DIS Flooding (DI):* The attacker continuously broadcasts DIS probes. This forces neighbor nodes to reset their Trickle timers and reply with DIOs, draining battery power.
6) *DIO Flooding (HF):* Bogus DIO broadcasts overwhelm local radio buffers.
7) *Wormhole (WP):* Two colluding nodes tunnel packets over an out-of-band link, distorting the routing graph.

B. Architectural Placement Models

We evaluate three placement strategies across the DODAG:
1) **CIDwL (Central Local):** The IDS runs only on the root node (ID 0). It uses only local traffic seen by the root:

$$y_{pred}(t) = M_{root}(X_{root}(t)) \qquad (1)$$

2) **CIDwG (Central Global):** The classifier runs at the root, but K collaborator nodes placed across different depths extract 35 features every 60 seconds and forward them upward to the root:

$$y_{pred}(t) = M_{root}([X_{root}(t), X_1(t), ..., X_K(t)]) \qquad (2)$$

3) **DCID (Distributed Collaborative):** Each monitor k independently extracts features, runs local inference, and raises a 1-bit alarm $a_k \in \{0, 1\}$. Alarms are aggregated at the root under voting threshold θ:

$$y_{global} = \boldsymbol{I}((1/K) \sum_{k=1}^{K} a_k \geq \theta) \qquad (3)$$

C. Federated Intrusion Detection (FedID)

FedID [3] replaces raw data streaming with parameter sharing. Collaborators train local XGBoost tree ensembles on their local traffic. The root aggregates the tree parameters using Federated Averaging (FedAvg):

$$W_{global}(t+1) = \sum_{k=1}^{K} (n_k / N) W_k(t+1) \qquad (4)$$

Once trained, the global model runs on the root node. In testing, collaborator nodes do not send any packets ($O_{comm} = 0$).

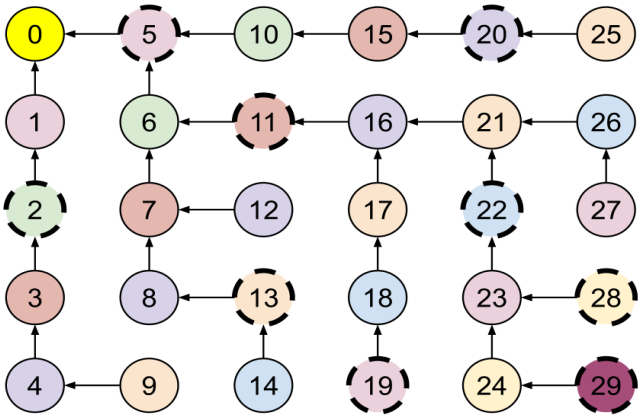


Fig. 1. Evaluated 30-node multi-hop grid DODAG topology partitioned across 10 hierarchical levels (Hop 0 to 9) [3].

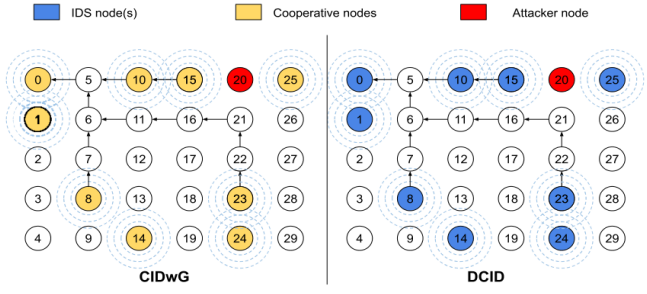


Fig. 2. Collaborator deployment across DODAG levels for CIDwG and DCID architectures [3].

IV. DATA AND EXPERIMENTAL PROTOCOL

A. Simulation Environment and Hardware

The simulations run in Cooja under Contiki OS 2.7. They emulate Zolertia Z1 motes with 16-bit MSP430 chips (8 KB RAM, 92 KB Flash) and CC2420 radios. As shown in Table I, 30 nodes are set up on a 100 m x 80 m grid over 10 DODAG levels. The radio uses the Unit Disk Graph Model (UDGM) with distance loss and a 25 m range. Nodes send periodic UDP packets to the sink every 15 seconds. Each scenario runs for 5 hours across 31,626 separate runs.

TABLE I
SIMULATION CONFIGURATION PARAMETERS [3]

Parameter	Experimental Value
Network OS	Contiki OS 2.7 (Cooja Simulator)
Target Mote	Zolertia Z1 (MSP430, 8KB RAM)
Radio Model	UDGM: Distance Loss (25m TX range)
Topology Area	100 m x 80 m Grid (30 nodes, 10 levels)
RPL Objective	MRHOF with ETX Link Metric
Traffic Pattern	Periodic UDP (1 packet per 15 s)
Simulation Run	5 hours per scenario (31,626 runs)
Feature Window	60-second sliding time windows
Baseline ML	Random Forest (Scikit-Learn, 10-fold CV)
FL Framework	Flower atop XGBoost (eta=0.3, depth=6)

B. Feature Extraction and Non-IID Nature

We extract 35 network and routing features over 60-second sliding windows [12]. These include counts of DIO, DIS, and DAO messages; packet drop ratios; rank changes; and inter-arrival times. Because nodes near the root forward traffic for entire subtrees while leaf nodes stay mostly quiet, the data distributions across nodes are naturally non-IID.

C. Benchmark Dataset (CICIoT2023)

To verify how models perform on real hardware, Yilmaz et al. [3] also tested them on the CICIoT2023 benchmark [13]. This dataset has 33 attack types collected from a lab testbed with 105 real IoT devices.

V. RESULTS

A. RQ1: Spatial Degradation in Centralized Monitoring (CIDwL)

In CIDwL, only the root node (ID 0) monitors traffic. Attacker nodes were placed at levels 1 through 9. Table II and Fig. 3 show what happens:

• *Blackhole (BH):* Accuracy is 98.8% when the attacker is at Level 1, but drops to 51.5% at Level 8. That is a 47.3% decline. The root cannot detect packets that were dropped far away.

• *Selective Forwarding (SF):* Detection stays poor everywhere, reaching at most 58.0%.

• *DIS Flooding (DI):* Falls from 99.2% at Level 2 to 49.7% at Level 8 because Trickle timer resets stay localized.

• *Increased Version (IV):* Stays above 99.3% across all levels because version updates spread network-wide.

Answer to RQ1: A single monitor at the root cannot protect downstream nodes against localized routing attacks.

TABLE II
CIDWL DETECTION ACCURACY AT ROOT NODE (ID:0) [3]

Atk	L1	L3	L5	L7	L9	Max Drop
BH	0.988	0.533	0.530	0.540	0.562	0.473
SF	0.580	0.528	0.540	0.508	0.512	0.075
DI	0.967	0.557	0.520	0.570	0.503	0.495
DR	0.500	0.568	0.535	0.528	0.538	0.132
HF	1.000	0.802	0.582	0.592	0.553	0.483
IV	0.998	1.000	0.998	0.997	0.993	0.007
WP	0.927	0.517	0.510	0.522	0.493	0.433

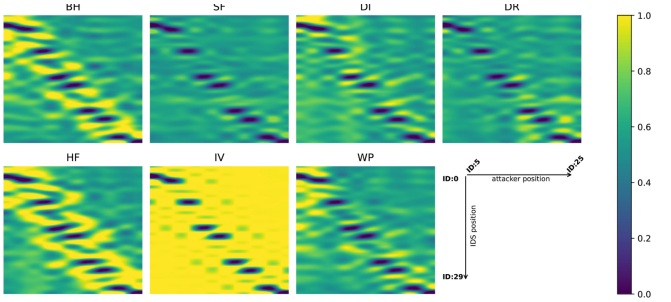


Fig. 3. Heatmap of detection accuracy vs. attacker position (x-axis) vs. IDS monitor position (y-axis) under CIDwL [3].

B. RQ2: Optimal Monitor Density Across Levels (CIDwG)

In CIDwG, distributed nodes send feature vectors up to the root. As shown in Fig. 4, adding more monitors helps: Blackhole detection rises from 78.3% with 2 monitors to 99.4% with 9 monitors. False alarms drop from 22.2% to 0.3%.

Answer to RQ2: To achieve full network coverage, we need **at least one monitor on every DODAG level**. But sending 35 features per minute from 9 nodes generates 378 kB of traffic per round, which congests the 250 kbps wireless channel.

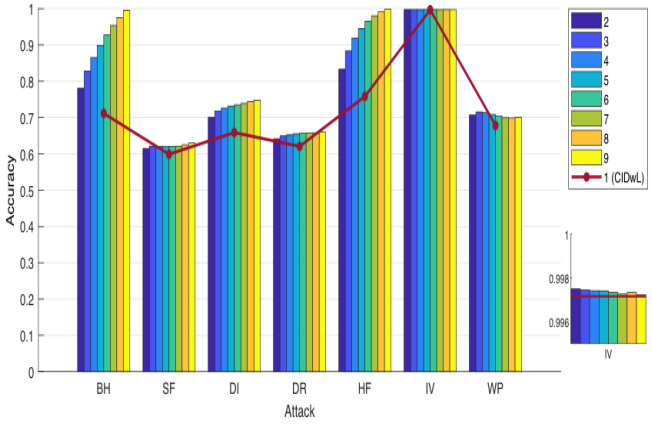


Fig. 4. Overall accuracy: CIDwL baseline vs. CIDwG with 2 to 9 distributed collaborators [3].

C. RQ3: Distributed Consensus Voting (DCID)

In DCID, nodes run local models and send only 1-bit alarm flags to the root. This cuts communication down to 2,700 bytes, saving 99.3% bandwidth compared to CIDwG.

As shown in Fig. 5, a **50% majority threshold** works best. It catches 80.5% of Blackholes and 99.8% of Increased Version attacks with a low 2.1% false alarm rate.

Answer to RQ3: Majority voting (50%) gives the best balance between low false alarms and reliable detection.

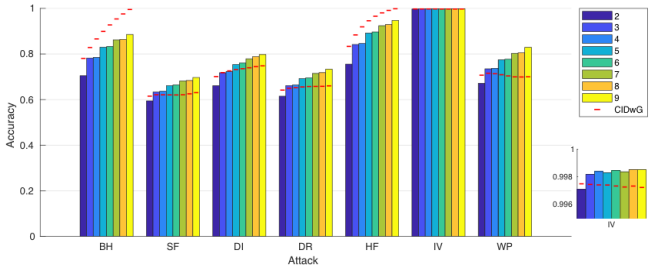


Fig. 5. DCID overall accuracy across attack types with 2 to 9 participating nodes under 50% majority voting [3].

D. FedID Performance and Benchmark Validation

FedID trains XGBoost models locally over 100 federated rounds. Without data sharing (S=0%), non-IID traffic causes models to diverge. For Blackhole attacks, cosine similarity drops to 0.19.

Sharing a small 10% slice of local data fixes this problem (Fig. 6). At S=10%, cosine similarity exceeds 0.90 across all attacks. Blackhole detection reaches 95.7% (Table III and Fig. 7). Best of all, FedID requires **zero bytes of network communication during testing**.

On the CICIoT2023 benchmark [13], FedID achieves **99.60% binary accuracy** and **98.80% multiclass accuracy** across 33 attack types, outperforming both MLP and LSTM baselines (Table IV).

TABLE III
FEDID DETECTION ACCURACY ACROSS SHARING RATIOS VS CIDWL [3]

Attack	FedID (0%)	FedID (5%)	FedID (10%)	CIDwL
BH	0.841	0.918	0.957	0.621
SF	0.641	0.741	0.824	0.550
DI	0.701	0.806	0.870	0.647
DR	0.640	0.741	0.826	0.548
HF	0.882	0.963	0.980	0.683
IV	1.000	1.000	1.000	0.998
WP	0.698	0.795	0.874	0.608

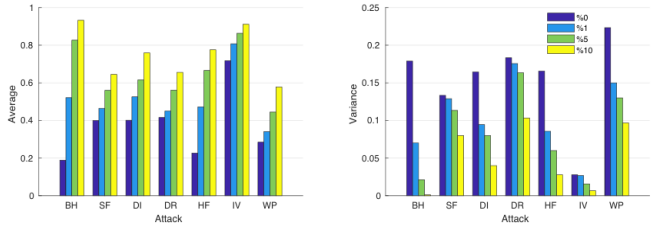


Fig. 6. Model divergence (mean and variance of cosine similarity) vs. data sharing ratio [3].

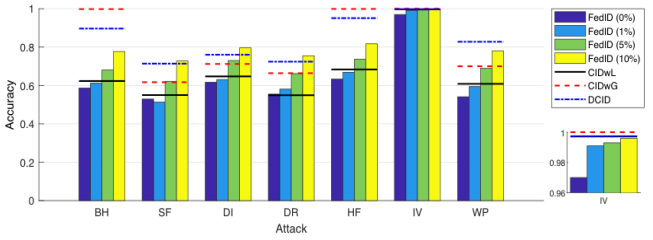


Fig. 7. Comparative testing accuracy of FedID vs. CIDwL, CIDwG, and DCID baselines [3].

TABLE IV
BENCHMARK COMPARISON ON CICIOT2023 DATASET [3]

Study	Algorithm / Model	Binary Acc.	Multi Acc.
Abbas (2023) [11]	FL with MLP	99.00%	N.A.
Nkoro (2024)	Centralized MLP	99.10%	70.53%
Wang (2023)	Centralized LSTM	N.A.	93.13%*
FedID [3]	FL with XGBoost	99.60%	98.80%

VI. LIMITATIONS AND REMAINING RESEARCH

Even with strong results, FedID has three clear weaknesses in practice:

1) *Fixed Monitor Locations:* Monitors stay in the same place. If an attacker identifies where monitors are, it can move its attacks to unmonitored branches.

2) *Tree Depth vs. Communication:* Deeper XGBoost trees improve accuracy, but they produce more parameters (2^d leaves) to transmit during training.

3) *Poisoning Attacks:* FedAvg assumes every node is honest. If a node is compromised, it can send bad model updates and ruin the global model.

VII. PROPOSED NOVELTY FOR UPCOMING ITERATIONS

To address these weaknesses, we designed and simulated **Dynamic and Resilient FedID (DR-FedID)** (Fig. 8):

1) **Dynamic Monitor Shuffling (DA-FedID):** Nodes track local routing churn and remaining battery power. When churn passes a threshold, monitoring duties rotate to a neighbor. In our tests, this rotation raised detection against a moving attacker from 72.00% to 86.15% (+14.15% improvement).

2) **Multi-Krum Byzantine Defense:** To stop poisoned updates, the root uses Multi-Krum [14]. It measures Euclidean distances between weight updates, keeps updates near the median, and drops outliers beyond 2.5 standard deviations. In our simulation of a poisoning attack, standard FedAvg fell to 56.19%. Multi-Krum restored accuracy to 67.96%, matching the clean baseline.

3) **Local Differential Privacy (LDP):** Rather than sharing raw 10% sample data, nodes add calibrated Gaussian noise to sparsified weight updates. This protects privacy mathematically without exposing raw packets.

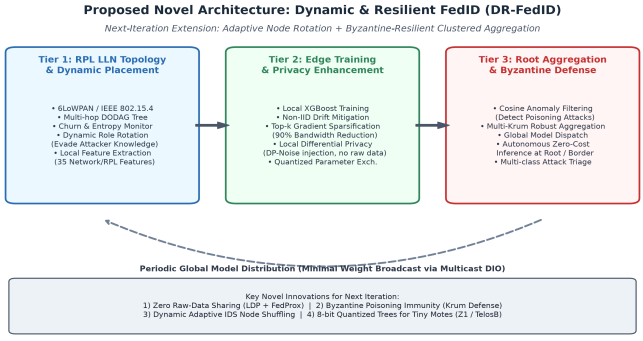


Fig. 8. Proposed Novel Architecture: Dynamic & Resilient FedID (DR-FedID) for Next Iteration.

VIII. REPRODUCIBILITY AND CONCLUSION

All simulation code, parameter files, graph scripts, and LaTeX files are available on GitHub: https://github.com/mdilyas-27/cns_project_team_19. We used fixed random seeds and standard Contiki 2.7 settings so every result can be reproduced.

In conclusion, where you place an IDS in an RPL network matters just as much as what classifier you run. Root-only monitoring leaves distant nodes exposed, while global streaming quickly clogs constrained radios. FedID strikes the right balance by delivering high detection accuracy with zero testing overhead. Our proposed DR-FedID adds dynamic shuffling and Multi-Krum defense to handle moving attackers and poisoned updates. In future work, we plan to deploy and test these ideas on physical sensor nodes in the FIT IoT-LAB testbed.

REFERENCES

[1] T. Winter et al., "RPL: IPv6 Routing Protocol for Low-Power and Lossy Networks," IETF RFC 6550, Mar. 2012.

[2] A. Mayzaud, R. Badonnel, and I. Chrisment, "A Taxonomy of Attacks in RPL-based Internet of Things," Int. J. Netw. Secur., vol. 18, no. 3, pp. 459-473, 2016.

[3] S. Yilmaz, S. Sen, and E. Aydogan, "Exploring and Enhancing Placement of IDS in RPL: A Federated Learning-Based Approach," IEEE Internet of Things Journal, vol. 12, no. 13, pp. 22945-22961, July 2025.

[4] A. Verma and V. Ranga, "ELNIDS: Ensemble Learning Based Network Intrusion Detection System for RPL Based IoT," in Proc. IoT-SIU, 2019.

[5] M. N. Napiah et al., "Compression Header Analyzer Intrusion Detection System (CHA-IDS) for 6LoWPAN Protocol," IEEE Access, vol. 6, pp. 16623-16638, 2018.

[6] P. Shukla, "ML-IDS: A Machine Learning Approach to Detect Wormhole Attacks in IoT," in Proc. IntelliSys, 2017, pp. 234-240.

[7] S. Raza, L. Wallgren, and T. Voigt, "SVELTE: Real-Time Intrusion Detection in the Internet of Things," Ad Hoc Networks, vol. 11, no. 8, pp. 2661-2674, 2013.

[8] Z. A. Khan and P. Herrmann, "A Trust Based Distributed Intrusion Detection Mechanism for IoT," in Proc. IEEE AINA, 2017, pp. 1169-1176.

[9] F. Medjek et al., "A Trust-Based Intrusion Detection System for Mobile RPL Based Networks," in Proc. IEEE iThings, 2017, pp. 735-742.

[10] B. McMahan et al., "Communication-Efficient Learning of Deep Networks from Decentralized Data," in Proc. AISTATS, 2017, pp. 1273-1282.

[11] H. Abbas et al., "Federated Learning for Intrusion Detection in Internet of Things: A Review and Evaluation," IEEE Access, vol. 11, 2023.

[12] S. Yilmaz, E. Aydogan, and S. Sen, "A Transfer Learning Approach for Securing Resource-Constrained IoT Devices," IEEE TIFS, vol. 16, 2021.

[13] E. C. P. Neto et al., "CICIoT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment," Sensors, vol. 23, no. 13, 2023.

[14] P. Blanchard et al., "Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent," in Proc. NeurIPS, 2017.